

Aplicação para Segmentação de Ventrículos e Diagnostico Preventivo de Alzheimer usando imagens medicas

Arthur Castro Machado, Arthur Otávio Souza, João Gabriel Polonio,
Pedro Miranda Rodrigues

19 de novembro de 2025

Resumo

A Doença de Alzheimer é uma condição neurodegenerativa que provoca alterações volumétricas em estruturas cerebrais, perceptíveis em exames de ressonância magnética. Com o uso de processamento de imagens e aprendizado de máquina, é possível auxiliar no diagnóstico precoce e na caracterização de mudanças nas estruturas cerebrais. Este trabalho implementa e avalia uma solução computacional para segmentar os ventrículos laterais e aplicar métodos de classificação e regressão para auxiliar no diagnóstico de Doença de Alzheimer e estimar a idade de pacientes, utilizando um subconjunto do dataset OASIS 2.

1 Introdução

A Doença de Alzheimer (DA) é a forma mais comum de demência em idosos, caracterizando-se por um declínio progressivo da função cognitiva [1], tornando assim o diagnóstico precoce em um desafio clínico significativo, mas fundamental para o planejamento de intervenções que possam retardar a progressão da doença e melhorar a qualidade de vida do paciente.

O avanço das técnicas de neuroimagem, em particular a Ressonância Magnética (RM), tem se mostrado promissor na identificação de deformações associados à DA. Uma das alterações mais evidentes é a atrofia cerebral, que frequentemente se manifesta como um alargamento dos ventrículos laterais devido à perda de tecido neural circundante [2], sendo assim a quantificação precisa dessa alteração volumétrica pode servir como um indicador importante no auxílio ao diagnóstico e monitoramento da doença.

No entanto, a análise manual dessas estruturas em imagens de RM é um processo laborioso, demorado e suscetível à variabilidade entre diferentes observadores. Nesse contexto, o Processamento de Imagens e o Aprendizado de Máquina oferecem ferramentas poderosas para automatizar e objetivar essa análise.

Este trabalho, desenvolvido no âmbito da disciplina de Processamento e Análise de Imagens, propõe o desenvolvimento de uma solução computacional gráfica para auxiliar nesse desafio. O sistema implementado foca em duas tarefas principais, utilizando um subconjunto de imagens axiais do dataset OASIS 2:

- **Segmentação e Classificação:** Implementação de um método automático para segmentar os ventrículos laterais e extrair características morfológicas (como área, circularidade e excentricidade). Essas características são, então, utilizadas para treinar modelos de aprendizado de máquina rasos para classificar pacientes entre "Dementes" e "Não Dementes".
- **Análise por Aprendizado Profundo:** Aplicação de modelos de Aprendizado Profundo, especificamente a arquitetura EfficientNet, utilizando as imagens de RM como entrada direta para realizar tanto a classificação de demência quanto a regressão da idade do paciente.

O objetivo é comparar a eficácia das abordagens baseadas em extração de features com as abordagens de aprendizado profundo no contexto do diagnóstico de DA e estimativa de idade.

2 Implementação

2.1 Ferramentas e Bibliotecas Utilizadas

O desenvolvimento da solução foi realizado integralmente na linguagem Python, escolhida devido à sua robustez e ao vasto ecossistema de bibliotecas voltadas para ciência de dados e processamento de imagens médicas. As principais ferramentas empregadas no projeto foram:

- **NumPy e Pandas:** Utilizadas para a manipulação de estruturas de dados fundamentais. O NumPy permitiu operações vetoriais eficientes sobre as matrizes de pixels das imagens, enquanto o Pandas foi essencial para o gerenciamento e estruturação dos metadados clínicos (CSV) e das características extraídas.
- **scikit-image:** Biblioteca de processamento de imagens utilizada na construção do pipeline de segmentação. Forneceu as implementações para o realce de contraste (CLAHE), filtros de suavização e operações morfológicas matemáticas necessárias para isolar os ventrículos.
- **scikit-learn:** Empregada para algoritmos de aprendizado de máquina clássico e utilitários de avaliação. Foi utilizada na implementação do algoritmo K-Means (para segmentação de tecidos), da Regressão Linear (para classificação) e nas funções de divisão estratificada de dados (*train/test split*) e cálculo de métricas.
- **XGBoost:** Biblioteca de *Gradient Boosting* otimizada, baseada em árvores de decisão. Foi a ferramenta escolhida para a tarefa de regressão da idade a partir de dados tabulares, devido à sua eficiência em lidar com dados estruturados.
- **TensorFlow e Keras:** Frameworks de aprendizado profundo (*Deep Learning*) utilizados para a construção, treinamento e avaliação dos modelos de redes neurais convolucionais, especificamente na implementação da arquitetura EfficientNetB0.
- **NiBabel:** Biblioteca especializada no acesso a formatos de neuroimagem. Foi responsável pela leitura e interpretação dos arquivos volumétricos no formato NIFTI (.nii.gz) provenientes do dataset OASIS-2.
- **PySide6:** Binding oficial da biblioteca Qt para Python. Foi utilizada para o desenvolvimento da interface gráfica do usuário (GUI), permitindo o carregamento interativo de exames e a visualização dos resultados das segmentações.
- **Matplotlib e Seaborn:** Bibliotecas de visualização de dados utilizadas para a geração de gráficos analíticos, incluindo as curvas de aprendizado do treinamento profundo, matrizes de confusão e gráficos de dispersão para análise de regressão.

2.2 Segmentação

A segmentação dos ventrículos laterais foi implementada por meio de um pipeline de múltiplas etapas aplicadas na camada central da fatia axial do exame.

2.2.1 Pré-processamento da Imagem

Para otimizar a distinção entre os tecidos, são aplicadas três etapas de pré-processamento:

1. **Normalização:** As intensidades dos pixels da imagem de entrada são normalizadas para o intervalo de ponto flutuante $[0, 1]$.
2. **Realce de Contraste:** É aplicada a técnica de Equalização de Histograma Adaptativa Limitada por Contraste (CLAHE). Esta técnica melhora o contraste local da imagem, realçando as bordas entre diferentes tipos de tecido cerebral (matéria cinzenta, matéria branca e líquido cefalorraquidiano).
3. **Suavização (Filtro Gaussiano):** Um filtro Gaussiano com desvio padrão (σ) de 1 é aplicado para reduzir o ruído de alta frequência presente na imagem de RM, o que facilita as etapas subsequentes de limiarização e clusterização.

2.2.2 Extração Cerebral (Skull Stripping)

Para que a análise se concentre apenas no tecido cerebral, é realizada uma remoção do crânio [3]. O processo utiliza o método de limiarização de Otsu sobre a imagem suavizada para encontrar um limiar global ótimo que separe o cérebro e o crânio do fundo da imagem. A máscara binária resultante é refinada por meio de operações morfológicas:

- Uma **abertura binária** (erosão seguida de dilatação) com um elemento estruturante em forma de disco é usada para remover pequenas áreas brilhantes e ruídos.
- A função `remove_small_objects` é usada para eliminar quaisquer componentes conectados com área inferior a 5000 pixels.
- A função `binary_fill_holes` preenche buracos internos na máscara.

Finalmente, apenas o **maior componente conectado** desta máscara é mantido, resultando em uma máscara robusta do parênquima cerebral usada para extrair a região de interesse da imagem pré-processada.

2.2.3 Identificação do Líquido Cefalorraquidiano (LCR)

Com o cérebro isolado, identificam-se os pixels correspondentes ao Líquido Cefalorraquidiano (LCR), que preenche os ventrículos. Isso é feito por meio de clusterização:

1. O algoritmo **K-Means** é aplicado sobre os valores de intensidade de todos os pixels pertencentes à máscara cerebral.
2. O número de clusters foi definido como $k = 4$, com o objetivo de agrupar os pixels nas classes principais: LCR, matéria cinzenta, matéria branca e, possivelmente, uma classe de volume parcial.
3. Assume-se que o LCR, por ser o tecido com menor intensidade (mais escuro) na imagem de RM ponderada em T1, corresponde ao cluster com o **centróide de menor valor**.

Uma máscara binária é criada contendo todos os pixels classificados pelo K-Means como pertencentes a este cluster de LCR.

2.2.4 Isolamento dos Ventrículos

A máscara de LCR total contém tanto os ventrículos laterais quanto o LCR presente nos sulcos cerebrais. Para isolar apenas os ventrículos, foi implementada uma filtragem baseada em propriedades topológicas e morfológicas:

1. **Transformada de Distância:** É calculada a Transformada de Distância Euclidiana sobre a `mask_lcr_total`. Esta operação atribui a cada pixel de LCR um valor correspondente à sua distância até a borda mais próxima. Regiões internas e "espessas", como os ventrículos, terão valores de distância mais altos no seu interior, enquanto regiões finas, como os sulcos, terão valores baixos.
2. **Análise de Componentes Conectados:** A imagem da transformada de distância é rotulada para identificar todas as regiões de LCR conectadas.
3. **Filtragem de Regiões:** Cada região de LCR é então analisada e filtrada com base em dois critérios heurísticos:
 - **Tamanho (Área):** A área da região deve estar entre 100 e 15000 pixels. Isso remove pequenos artefatos de ruído (muito pequenos) e possíveis falhas de segmentação que englobam todo o LCR (muito grandes).
 - **Centralidade:** O centróide da região deve estar localizado próximo ao centro geométrico da imagem (dentro de uma tolerância de 20% da dimensão da imagem). Esta heurística se baseia na localização anatômica esperada dos ventrículos laterais, descartando os sulcos, que são periféricos.

As regiões que satisfazem ambos os critérios são consideradas parte dos ventrículos laterais e são adicionadas à máscara de segmentação final. Por fim, uma última operação de `binary_fill_holes` é aplicada para garantir que as regiões ventriculares segmentadas sejam sólidas e não contenham buracos.

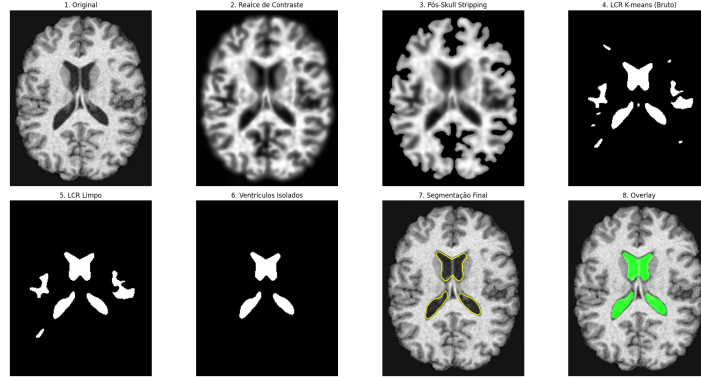


Figura 1: Exemplo de Segmentação

2.3 Extração de Descritores

Após a obtenção da máscara binária dos ventrículos se faz a caracterização quantitativa dessas regiões, são extraídos descritores morfológicos para quantificar a geometria das estruturas segmentadas:

1. **Área (Ventricle_Area):** Descritor obrigatório. Corresponde à área total da máscara, calculada como a soma do número de pixels de todos os componentes ventriculares detectados. É o indicador mais direto da expansão ventricular.
2. **Circularidade (Ventricle_Circularity):** Descritor obrigatório. Mede o quão próxima à forma de uma região está de um círculo perfeito (valor 1.0). Para formas com múltiplos componentes, foi calculada a média da circularidade de cada componente.
3. **Excentricidade (Ventricle_Eccentricity):** Descritor obrigatório. Mede o "achatamento" da forma, variando de 0 (um círculo) a 1 (uma linha). Foi calculada como a média da excentricidade de todos os componentes.
4. **Perímetro (Ventricle_Perimeter):** Primeiro descritor de livre escolha. Representa a soma do comprimento do contorno (em pixels) de todos os componentes ventriculares.
5. **Solidez (Ventricle_Solidity):** Segundo descritor de livre escolha. Mede a proporção de pixels da região em relação ao seu fecho convexo (*convex hull*). Valores próximos de 1 indicam uma forma convexa, sem concavidades. Foi utilizada a média da solidez de todos os componentes.
6. **Comprimento do Eixo Principal (Ventricle_MajorAxisLength):** Terceiro descritor de livre escolha. Mede o comprimento do eixo mais longo da elipse que melhor se ajusta à região. Foi calculada a média desta medida entre todos os componentes.

2.4 Tratamento de Dados

A base de dados utilizada foi um subconjunto do projeto OASIS-2 (*Open Access Series of Imaging Studies*) que contem exames longitudinais de ressonância magnética [2], sendo usado especificamente os exames do arco axial.

Inicialmente foi realizado o processamento em lotes de todas as 373 imagens presentes na base. Para cada imagem foram executados os algoritmos de segmentação e extração de features descritos na seção anterior. Os dados resultantes foram fundidos (*merge*) com os dados demográficos e clínicos originais do projeto OASIS-2, utilizando o identificador único do exame (MRI ID) como chave. Durante esse processo se cria o CSV `feature_full` que além de usar os dados do *merge*, os limpa para possuir apenas os identificadores de Diagnóstico e Idade, os Ids de Identificação e por fim as features retiradas da segmentação.

Durante a etapa de *merge*, foi realizado uma limpeza e padronização das classes alvo. Para transformar a base de dados em binária, uma vez que inicialmente ela possui as classes *NonDemented*, *Demented* e *Converted*, fez-se um tratamento específico para o grupo 'Converted':

- **Grupo Demented:** Mantido como *Demented* (Classe 1).
- **Grupo Converted:** Mapeado como *Demented* (Classe 1) apenas se o valor de CDR fosse maior que 0 na visita específica. Caso contrário (CDR=0 ou nulo), foi mapeado como *NonDemented* (Classe 0).
- **Grupo Nondemented:** Mapeado como *NonDemented* (Classe 0).

Por fim, o CSV e a base de dados de imagens são particionadas levando em conta os pacientes, não os exames realizados, seguindo a seguinte lógica de agrupamento:

1. **Agrupamento:** Os dados foram agrupados por *Subject ID*. Para fins de estratificação, atribuiu-se a cada paciente o rótulo de "maior gravidade" observado. Se um paciente apresentou demência em qualquer visita (Classe 1), ele foi considerado Demente para o sorteio da divisão (CDR maior que 0).
2. **Divisão (Split):** O conjunto de 150 pacientes únicos foi dividido utilizando a função `train_test_split` do *scikit-learn* com estratificação:
 - **Teste:** 20% dos pacientes (30 sujeitos) foram separados isoladamente.
 - **Validação:** Dos 80% restantes, separou-se 20% (24 sujeitos) para validação.
 - **Treino:** Os demais pacientes (96 sujeitos) compuseram o conjunto de treino.

Após a definição dos grupos de pacientes, todos os exames (visitas) associados a esses indivíduos foram movidos fisicamente para diretórios específicos (`treino/`, `validacao/` e `teste/`), juntamente com seus respectivos arquivos CSV filtrados (`features_full.csv`), garantindo que o modelo não tenha contato com qualquer imagem de um paciente do conjunto de teste durante o treinamento.

2.5 Modelagem Baseada em Aprendizado Raso

Diferente da abordagem de Aprendizado Profundo, que processa a imagem bruta, a etapa de aprendizado raso utiliza o vetor de características extraído na Seção 2.2 (Área, Perímetro, Circularidade, Excentricidade, Solidez e Eixo Maior) como entrada. O objetivo é avaliar se esses biomarcadores geométricos, isoladamente, possuem poder preditivo suficiente para o diagnóstico e a estimativa de idade.

2.5.1 Classificação de Grupo: Regressão Linear com Limiar Otimizado

Para a classificação binária entre os grupos *NonDemented* (0) e *Demented* (1), optou-se por uma abordagem analítica baseada em Regressão Linear Múltipla. Embora logicamente menos comum que a Regressão Logística, esta abordagem foi adaptada para classificação através de um ajuste fino do limiar de decisão (*threshold tuning*). O modelo estima um valor contínuo de propensão $\hat{y}$ para cada paciente com base nas características morfológicas x :

$$\hat{y} = w^T x + b \quad (1)$$

Onde w representa os pesos atribuídos a cada descritor (como a importância da área ventricular) e b o viés. Para converter essa saída contínua em uma classe binária, foi implementado um algoritmo de busca exaustiva no conjunto de validação:

1. O modelo é treinado no conjunto de treino para minimizar o erro quadrático.
2. No conjunto de validação, varia-se um limiar τ no intervalo $[0.0, 1.0]$ com passo de 0.01.

3. Para cada τ , a classe é predita como:

$$C = \begin{cases} 1(\text{Demented}) & \text{se } \hat{y} \geq \tau \\ 0(\text{NonDemented}) & \text{se } \hat{y} < \tau \end{cases} \quad (2)$$

4. O valor de τ que maximiza a acurácia na validação é salvo e utilizado para as predições finais no conjunto de teste.

2.5.2 Regressão de Idade: XGBoost

Para a estimativa da idade biológica com base na atrofia ventricular, utilizou-se o algoritmo XGBoost (*Extreme Gradient Boosting*). Este modelo é baseado em árvores de decisão e utiliza o gradiente descendente para minimizar erros de forma iterativa, sendo o estado da arte para dados tabulares estruturados.

Os hiperparâmetros do modelo foram definidos empiricamente para evitar *overfitting* dada a dimensão reduzida do vetor de características:

- **Objetivo:** Regressão com erro quadrático (`reg:squarederror`).
- **Número de estimadores:** 500 árvores.
- **Taxa de aprendizado (*learning rate*):** 0.05.
- **Profundidade máxima:** 5 níveis.

A métrica de avaliação adotada para este regressor foi o Erro Médio Absoluto (MAE), que mensura a diferença média em anos entre a idade real e a idade predita pela morfologia dos ventrículos.

2.6 Classificador Profundo

Um classificador profundo é um modelo de aprendizado supervisionado baseado em redes neurais artificiais com múltiplas camadas. Esse tipo de arquitetura pertence ao campo do Aprendizado Profundo (Deep Learning), que se destaca por sua capacidade de identificar padrões complexos em dados de alta dimensionalidade. O objetivo de um classificador profundo é aprender uma função capaz de mapear uma entrada, como uma imagem, um vetor numérico ou um conjunto de atributos para uma categoria específica.

A principal vantagem dos classificadores profundos é sua habilidade de extrair automaticamente as características relevantes do dado bruto. Em vez de depender de atributos construídos manualmente, a própria rede aprende o que deve avaliar:

- Camadas iniciais identificam padrões simples, como contrastes, bordas, pequenas regiões de interesse.
- Camadas intermediárias combinam essas informações em estruturas mais abstratas, como formas ou texturas.
- Camadas finais consolidam essas representações para distinguir classes específicas.

Esse processo torna os modelos profundos extremamente eficazes em problemas onde há muitos atributos, grande variabilidade entre as amostras ou relações internas difíceis de serem descritas manualmente. Quanto maior a complexidade e o volume dos dados, mais evidente é a superioridade do aprendizado profundo sobre métodos tradicionais.

Modelos profundos também conseguem lidar com grandes quantidades de dados e aproveitar padrões estatísticos que só são perceptíveis quando milhares de exemplos são analisados.

Ao contrário de classificadores clássicos, redes profundas podem aprender relações não lineares altamente complexas sem exigir pré-processamentos extensos. Isso as torna adequadas para cenários onde:

- A entrada possui milhares de atributos.

- Existe correlação entre variáveis difíceis de serem descritas.
- As classes são separadas por padrões sutis.
- Os dados têm estrutura espacial (como imagens) ou temporal.

2.6.1 Pré-processamento das imagens NIfTI

Para cada arquivo, a mesma matriz bidimensional é replicada nos três canais, resultando em uma imagem de dimensão $(H, W, 3)$, em que cada canal contém exatamente a mesma informação de intensidade.

Após a construção da imagem, é aplicada uma normalização de intensidade por máximo, a fim de reduzir diferenças de escala entre exames. De forma que os valores resultantes fiquem aproximadamente no intervalo $[0, 1]$.

Na sequência, cada imagem é redimensionada para uma resolução fixa por meio de interpolação bilinear, adequando-se assim ao tamanho de entrada esperado pela EfficientNet-B0. Por fim, é aplicada a função de pré-processamento nativa da arquitetura, que reescala os valores de pixel para a faixa aproximadamente $[-1, 1]$ e realiza as transformações adicionais necessárias para alinhar os dados ao espaço de características aprendido no pré-treinamento.

Ao término dessa sequência, cada exame NIfTI bidimensional é representado por uma imagem com resolução fixa e intensidades compatíveis com o backbone convolucional utilizado na etapa de classificação.

2.6.2 Construção do conjunto de dados

A organização do conjunto de dados e o fluxo de pré-processamento durante o treinamento foram implementados utilizando a API `tf.data`. A partir de um *dataframe* contendo os caminhos para os arquivos NIfTI e os rótulos binários (correspondendo às classes *NonDemented* e *Demented*), é criado um *dataset*.

Quando o conjunto é utilizado em modo de treinamento, os exemplos são embaralhados com um *buffer* do tamanho do próprio conjunto, utilizando uma semente fixa 42. Em seguida, cada linha do dataset é transformada em (*imagem_processada*, *rótulo*) pela função de pré-processamento descrita na subseção anterior.

Com o objetivo de reduzir o overfitting e aumentar a capacidade de generalização do classificador, é aplicado um bloco de *data augmentation* apenas ao conjunto de treinamento. Esse bloco é implementado como um modelo sequencial em Keras, contendo as seguintes transformações:

- `RandomFlip("horizontal")`
- `RandomRotation(0.05)`
- `RandomZoom(0.05)`

Essas operações são aplicadas a cada mini-lote durante o treinamento, de modo que o modelo é exposto a diferentes versões de uma mesma imagem ao longo das épocas, enriquecendo o conjunto efetivo de exemplos vistos pela rede.

2.6.3 EfficientNet

A EfficientNet é uma família de modelos de CNN projetada para alcançar alta acurácia com custo computacional reduzido, introduzida por pesquisadores do Google AI em 2019 [4] [5] [6]. O modelo foi projetado para otimizar simultaneamente o desempenho e a eficiência computacional, estabelecendo um novo paradigma no desenvolvimento de arquiteturas profundas.

2.6.3.1 Arquitetura-base: EfficientNet-B0

A arquitetura inicial (B0), obtida por meio de Neural Architecture Search (NAS), combina operações altamente eficientes, como os blocos MBConv com mecanismos squeeze-and-excitation, garantindo boa capacidade de extração de características com um número reduzido de parâmetros. A partir dela, surgem as variantes (B0 a B7) da EfficientNet[6]

2.6.3.2 Escalonamento Composto

Uma dos principais diferenciais da EfficientNet é o método de Compound Scaling [6], que realiza o escalonamento conjunto da profundidade, largura e resolução da rede. Diferente de arquiteturas tradicionais que tipicamente aumentam apenas um desses fatores, o escalonamento composto mantém um equilíbrio global entre a complexidade computacional e a capacidade de representação. Essa abordagem permite que a EfficientNet alcance melhorias significativas em acurácia, preservando custos computacionais muito menores. [6].

2.6.3.3 Vantagens da EfficientNet

- Acurácia superior em comparação a arquiteturas como ResNet, DenseNet e Inception.
- Redução substancial no número de parâmetros e operações convolucionais.
- Excelente eficiência computacional, tornando-a apropriada tanto para dispositivos móveis quanto para aplicações em larga escala.
- Elevada estabilidade durante o treinamento, com menor propensão a overfitting em bases menores.
- Forte desempenho em cenários de transfer learning, o que amplia seu uso em aplicações biomédicas, classificação especializada e diagnósticos automatizados.

2.6.3.4 Implementação

A implementação deste classificador baseia-se na EfficientNet-B0 pré-treinada no ImageNet. Inicialmente, a rede base foi carregada sem as camadas superiores e com os pesos congelados. Esse cabeçalho é composto por uma camada de *Global Average Pooling*, seguida de uma camada totalmente conectada com 128 neurônios, ativação ReLU e regularização L2, além de camadas de *Dropout* (0,4) para tentar diminuir o overfitting. A saída é formada por um único neurônio com ativação sigmoide para classificação binária. O treinamento inicial foi feito com otimizador Adam, função de perda *binary cross-entropy* e um *Early Stopping* com restauração dos melhores pesos. Então, foi feito um *fine-tuning*, em que as camadas superiores da EfficientNet foram descongeladas parcialmente, preservando aproximadamente dois terços iniciais da rede congelados, de modo a evitar degradação das representações já treinadas. O modelo foi recompilado com uma taxa de aprendizado reduzida para garantir atualizações mais estáveis. O *fine-tuning* foi realizado com 15 épocas a mais, o mesmo *Early Stopping*. Após diversas tentativas, essas foram as características finais que permitiram que chegássemos no resultado obtido. De forma geral o modelo utiliza os seguintes hiper-parâmetros

- IMG_SIZE = 224
- BATCH_SIZE = 16
- EPOCHS = 20
- FINE_TUNE_EPOCHS = 15
- LEARNING_RATE = 0.0001
- LEARNING_RATE_FT = LEARNING_RATE / 10

2.6.3.5 Regressão da idade

Para a tarefa de regressão da idade cronológica, foi reutilizado o mesmo pipeline de pré-processamento de imagens descrito anteriormente. A diferença principal está na preparação dos rótulos: a idade dos pacientes, originalmente em anos, foi normalizada para o intervalo $[0, 1]$ a partir dos valores mínimo e máximo observados no conjunto de treino. Essa versão normalizada foi utilizada como alvo da rede, enquanto a idade original foi mantida apenas para fins de avaliação.

A arquitetura de regressão é composta pela EfficientNet-B0, seguida de uma operação de *global average pooling*, uma camada de *dropout* e uma camada densa final com um único neurônio e ativação

sigmoide. Essa escolha de ativação força a saída a permanecer no intervalo $[0, 1]$, compatível com os rótulos normalizados, e a predição em anos é obtida posteriormente por meio da transformação inversa do min-max. O treinamento é realizado em duas etapas: inicialmente com o backbone congelado, e em seguida com todas as camadas da EfficientNet liberadas para *fine-tuning*, utilizando uma taxa de aprendizado reduzida.

3 Ambiente Computacional e Tempos de Execução

Todos os experimentos, desde o pré-processamento das imagens até o treinamento dos modelos, foram conduzidos em um *desktop* pessoal seguindo as configurações de hardware abaixo:

- **Processador (CPU):** Intel(R) Core(TM) i5-9600K @ 3.70GHz (6 núcleos físicos).
- **Memória RAM:** 16 GB.

Não foram utilizados aceleradores gráficos dedicados (GPUs) de nível servidor, o que demonstra a viabilidade de execução da solução proposta em computadores convencionais, especialmente para a abordagem de aprendizado raso.

3.0.1 Análise de Desempenho Temporal

Os tempos de execução variaram significativamente entre as abordagens. Os tempos médios observados estão detalhados a seguir:

- **Preparação da Base de Dados (≈ 2 minutos):** Esta etapa engloba a leitura dos arquivos NIfTI, o pipeline completo de segmentação (pré-processamento, Otsu, K-Means, morfologia) e a extração de features para todas as 373 imagens do *dataset*. O tempo reduzido deve-se à eficiência das operações vetoriais implementadas via *NumPy*.
- **Treinamento dos Modelos Rasos (Instantâneo):** O treinamento da Regressão Linear (classificação) e do XGBoost (regressão) é executado em menos de 1 segundo. Isso ocorre devido à baixa dimensionalidade dos dados de entrada, que consistem em apenas 6 atributos numéricos (*features* geométricas) por paciente, exigindo pouquíssimo poder de processamento para a convergência dos algoritmos.
- **Treinamento do Modelo Profundo (10 a 15 minutos):** O treinamento da arquitetura EfficientNetB0 exigiu entre 10 e 15 minutos para completar as épocas de treinamento e *fine-tuning*. Este aumento expressivo no tempo, em comparação ao modelo raso, justifica-se pela necessidade de realizar o *backpropagation* em milhões de parâmetros da rede neural e processar as matrizes de pixels das imagens brutas, demandando uso intensivo da CPU.

4 Resultados e Discussão

A avaliação experimental foi conduzida utilizando as métricas de Acurácia, Precisão, Revocação (*Recall*) e F1-Score para as tarefas de classificação, e o Erro Médio Absoluto (MAE) para a regressão de idade. Os resultados obtidos, em ambas as abordagens, refletem os desafios impostos pela restrição rigorosa de separação dos dados por paciente em um *dataset* reduzido.

4.1 Análise da Tarefa de Classificação

Tanto o modelo raso quanto o profundo falharam em estabelecer uma fronteira de decisão robusta entre pacientes dementes e não dementes.

4.1.1 Classificador Raso (Regressão Linear)

O modelo baseado em *features* geométricas apresentou um forte viés para a classe majoritária (*Non-Demented*). Conforme observado na Matriz de Confusão (Figura 2), o classificador rotulou quase todas as amostras como Classe 0. Isso indica que os descritores morfológicos isolados (como área e excentricidade), sem normalização volumétrica por crânio total, possuem alta variância intra-classe, impedindo a separação linear.

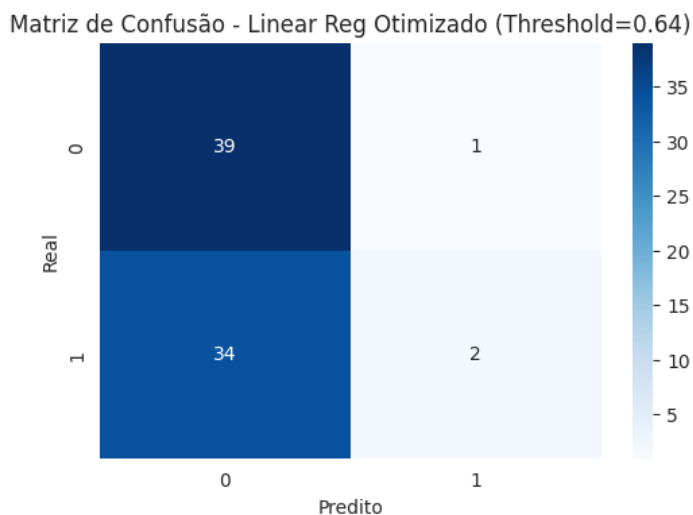


Figura 2: Matriz de Confusão do Classificador Raso. O modelo tornou-se um classificador trivial, optando quase exclusivamente pela classe majoritária (0).

4.1.2 Classificador Profundo (EfficientNet)

A rede neural, apesar de ter acesso à imagem bruta, sofreu de *overfitting* severo. As curvas de aprendizado (Figura 3) mostram um cenário divergente: enquanto a perda (*loss*) no treino cai consistentemente, a perda na validação sobe ou estagna. A acurácia final no teste oscilou próxima a 54%, marginalmente superior ao acaso, confirmando que a rede "memorizou" o conjunto de treino reduzido mas falhou em generalizar para novos pacientes.

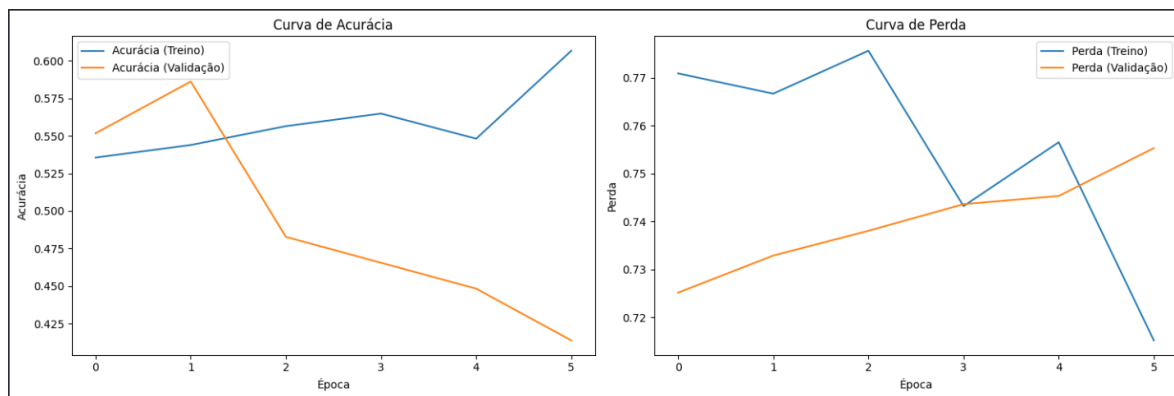


Figura 3: Curvas de treinamento da EfficientNet. A divergência entre as linhas azul (treino) e laranja (validação) evidencia o *overfitting* causado pela escassez de amostras.

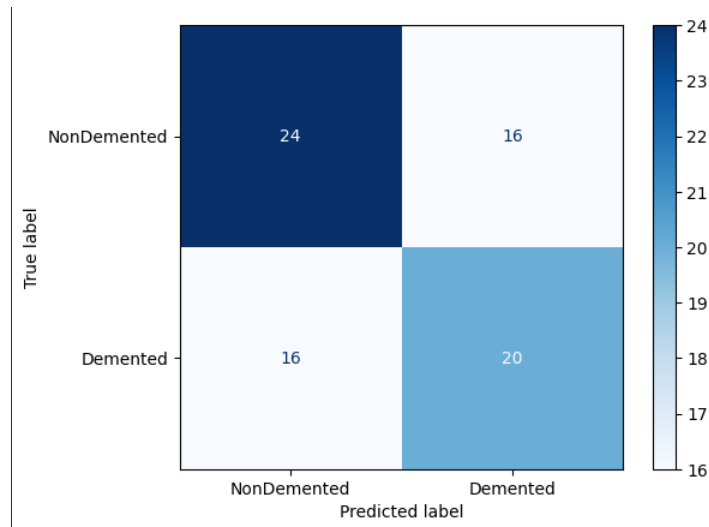


Figura 4: Matriz de Confusão do Classificador Profundo.

4.2 Análise da Tarefa de Regressão de Idade

Os modelos de regressão exibiram dois modos distintos de falha, ilustrados pelos gráficos de dispersão.

4.2.1 Regressão Rasa (XGBoost)

O modelo XGBoost apresentou previsões difusas e com baixa correlação com a idade real (Figura 5). Os pontos distribuem-se aleatoriamente em relação à reta ideal, indicando que as features geométricas extraídas não possuem correlação linear ou não-linear forte o suficiente com a idade cronológica neste conjunto de dados.

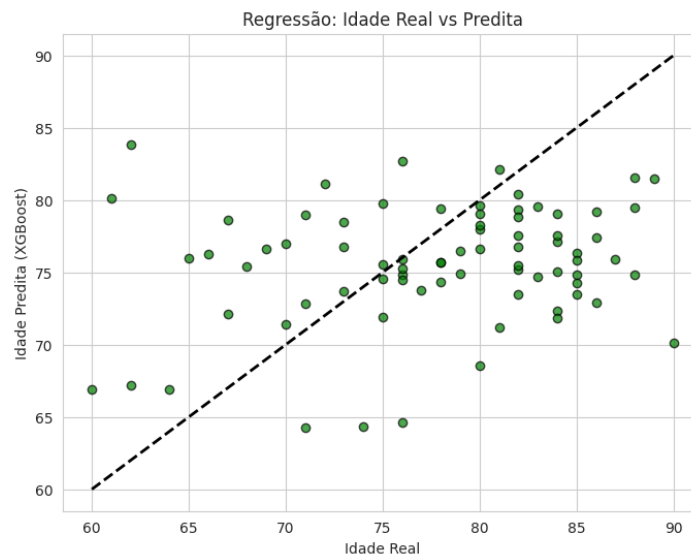


Figura 5: Dispersão Idade Real vs. Predita (XGBoost). A distribuição aleatória indica incapacidade de modelar o envelhecimento.

4.2.2 Regressão Profunda (EfficientNet)

Diferente do modelo raso, o modelo profundo exibiu um fenômeno conhecido como **regressão à média** ou colapso de predição. Como pode ser visto na Figura 6, o modelo prediz valores quase constantes

(uma linha horizontal próxima aos 80-81 anos) para todos os pacientes, independentemente se eles têm 60 ou 97 anos.

Um regressor contínuo seria mais adequadamente modelado com uma saída linear diretamente na escala da idade. A sigmoide foi utilizada para simplificar a implementação, porém introduz saturação nos extremos do intervalo e pode limitar a capacidade do modelo de representar idades muito baixas ou muito altas, o que se reflete nos resultados menos satisfatórios.

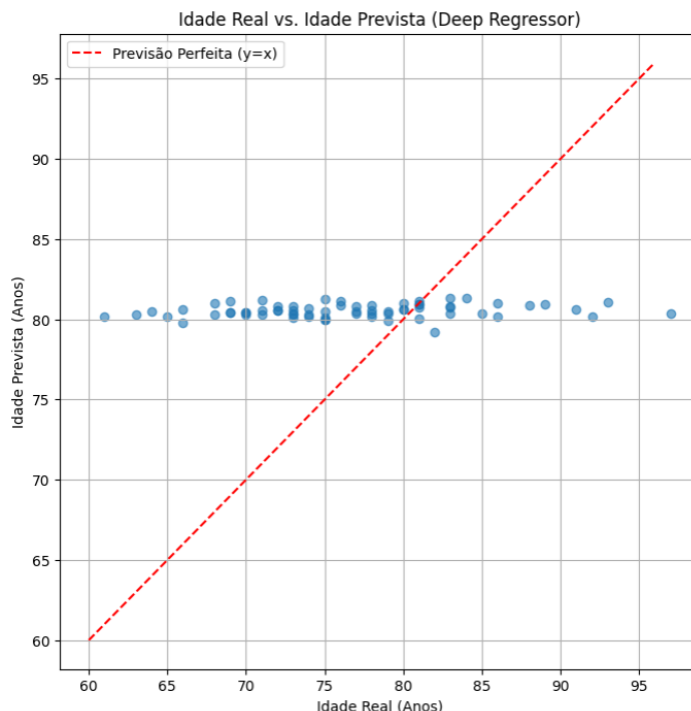


Figura 6: Dispersão Idade Real vs. Predita (Deep Learning). O modelo convergiu para uma solução trivial, predizendo a média de idade do conjunto de treino para todas as amostras, resultando em uma linha horizontal.

Isso ocorre quando a rede neural não consegue extrair *features* discriminativas da imagem para minimizar o erro. Para reduzir a função de perda (MAE/MSE) global, a rede aprende que "chutar a média" é a estratégia estatisticamente mais segura, resultando em um modelo sem utilidade clínica.

4.3 Discussão: O Impacto da Divisão por Paciente

A análise conjunta das quatro figuras aponta que a principal limitação não foi a escolha das arquiteturas, mas a **insuficiência de dados gerada pelo rigor metodológico**.

Optou-se neste trabalho por uma divisão de dados baseada estritamente em pacientes (*subject-based split*). Todos os exames de um indivíduo foram alocados exclusivamente em treino, validação ou teste. Essa abordagem evita o *data leakage* (onde o modelo decora a anatomia de um paciente), mas reduziu drasticamente o volume de treino efetivo para cerca de 96 sujeitos.

Com um número tão reduzido de exemplos únicos:

- O **XGBoost** não encontrou significância estatística nas variações sutis de geometria ventricular.
- A **EfficientNet**, que possui milhões de parâmetros, entrou em regime de *overfitting* na classificação e colapsou para a média na regressão, pois não havia variedade suficiente de anatomias para aprender a generalizar o envelhecimento ou a demência.

Conclui-se que a metodologia empregada é correta, mas a aplicação prática de Deep Learning neste contexto exige a expansão da base de dados para ordens de grandeza superiores a centenas de pacientes únicos.

Referências

- [1] World Health Organization, “Dementia,” March 2023.
- [2] D. S. Marcus and et al., “Open access series of imaging studies: longitudinal mri data in nondemented and demented older adults,” *Journal of cognitive neuroscience*, vol. 22, no. 12, pp. 2677–2684, 2010.
- [3] A. Hoopes, J. S. Mora, A. V. Dalca, B. Fischl, and M. Hoffmann, “Synthstrip: skull-stripping for any brain image,” *NeuroImage*, vol. 260, p. 119474, 2022.
- [4] T. Karmakar, “Introduction to efficientnet,” 2024.
- [5] M. Aborokbah, “Alzheimer’s disease mri classification using efficientnet: A deep learning model,” in *2024 4th International Conference on Applied Artificial Intelligence (ICAPAI)*, pp. 1–8, 2024.
- [6] M. Tan and Q. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” in *International conference on machine learning*, pp. 6105–6114, PMLR, 2019.